otris software

otrTest - Handbuch

Weitergabe und Vervielfältigung dieser Publikation oder von Teilen daraus sind, zu welchem Zweck und in welcher Form auch immer, ohne die ausdrückliche schriftliche Genehmigung durch die otris software AG nicht gestattet.

In dieser Publikation enthaltene Informationen können ohne vorherige Ankündigung geändert werden.

Alle in dieser Publikation aufgeführten Wort- und Bildmarken sind Eigentum der entsprechenden Hersteller.

Änderungen in der Software sind vorbehalten. Die in dieser Publikation enthaltenen Informationen stellen keinerlei Verpflichtung seitens des Verkäufers dar.

Aus Gründen der besseren Lesbarkeit wird auf eine geschlechtsneutrale Differenzierung verzichtet. Entsprechende Begriffe gelten im Sinne der Gleichbehandlung grundsätzlich für alle Geschlechter. Die verkürzte Sprachform beinhaltet keine Wertung. Dies gilt für die Begriffe in dieser Publikation ebenso wie für die Software selbst.

© Copyright 2025 otris software AG. Alle Rechte vorbehalten.

Inhaltsverzeichnis

Einleitung ... 3
- Verwendung ... 3

v3.0.0 ... 13

Getting started ... 13
- Einrichten per Documents CLI ... 13
- Einrichten in einem Projekt mit Tests ... 13

CLI-Befehle und Parameter ... 18
- Liste der verfügbaren Befehle ... 18
- Liste der verfügbaren Kommandozeilenparameter ... 18
- Konfiguration über Umgebungsvariablen ... 19

Mocha-Interface ... 20
- Hallo-Welt-Beispiel ... 20
- De-/Initialisierung der Testsuite ... 21
- Testausführung steuern ... 24
- Löschen der Testdaten deaktivieren ... 26

Vollständiges Beispiel ... 31

QuickCheck-Tests ... 31

Chai.JS ... 32
- Behavior-Driven Development ... 33
- Test-Driven Development ... 33

Tipps und Tricks ... 33
- Testausführung beenden ... 33
- Coverage-Daten von Gadgets-, Exits-, ScriptCall-Scripten und globale return-Statements ... 34

Einleitung

Framework für die Entwicklung von Tests in DOCUMENTS:

- Schreiben von Unit-Tests
- Scripting-Funktionen zum Prüfen erwartetem Verhalten (Werte, Exceptions usw.)
- Scripting-Funktionen zum Bereitstellen von Testdaten (Mappentypen, Vorgänge usw.)
- Scripting-Funktionen zum Aufräumen der Testdaten nach Testausführung
- Formate zur Darstellung und Auswertung der Testergebnisse (Text, JUnit)
- CLI für die Ausführung der Tests
- Automatische Ermittlung der Pfadabdeckung (Bericht)

Verwendung

```
$> documents test --help
```

v4.9.0

- Testergebnisse werden während der Ausführung einer Testsuite zwischengespeichert, so dass die auch noch nach einem Timeout oder anderweitigen Abbruch der Verbindung zum Server zur Verfügung stehen. Nach einem solchen Abbruch werden die Zwischenergebnisse vom Testframework heruntergeladen. (#68335)

v4.8.0

- Anpassungen für Auslieferung als SEA. (#69019)

v4.7.1

- Korrektur beim Löschen von Testdaten, wenn Testmappen bereits gelöscht waren. (#69018)

v4.7.0

- Neue Prüfungfunktionen `assertObjectsEqual()` und `assertFileFieldRights()`. (!481)
- Neuer Hook `preimport`, der vor dem Hochladen der Dateien, die in `include` angegeben sind, ausgeführt wird. (!480)
- Trat bei der Ausführung der Testsuiten ein Fehler auf, wurde nicht in allen Fällen der Verbindungs-Timeout zurückgesetzt. In Fällen, wo der Server nicht mehr verfügbar ist, konnte die

Fehlerbehandlung stark verzögert werden, wenn der `TESTSUITE_TIMEOUT` sehr hoch eingestellt wurde. (#67762)

v4.6.3

- `engines`-Eingabe in package.json korrigiert.

v4.6.2

- Abhängigkeit zu shelljs entfernt. (#67824)
- Löschen der Testdaten für bereits gelöschte Testdaten korrigiert, so dass keine falschen Fehlermeldungen ausgelöst werden. (#67178)

v4.6.1

- Fehler in Typings korrigiert.

v4.6.0

- Neue Hilfsfunktion `isDocFileDeleted()` zum prüfen, ob einen Mappenobjekt endgültig gelöscht wurde. (#63701)
- Erweiterung der Hilfsfunktionen `createSystemUser()` und `createFellow()` um die optionalen Parameter `licenseType` und `status`. (#66681)
- `createFellow()` schlug fehl, wenn kein Login angegeben wurde. Der generierte Default-Login-Name war zu lang. (#66681)
- Anpassung an erweiterte Prüfung im Server, so dass mit gelöschten Mappenobjekten nicht mehr gearbeitet werden kann. (#63701)
- Nutzerangabe auf der Kommandozeile wurde beim Kommando `test` nicht berücksichtigt. (#67347)
- Aktualisierung von documents-sds auf v2.5.1 (!474)
    - Wenn in Hooks Scripte ausgeführt wurden, die es nicht gibt, wurde kein Fehler ausgegeben.
    - Documents-Properties, die in der specs.json definiert waren (`properties`) wurden nicht richtig gesetzt.

v4.5.6

- Abhängigkeit @otr-tools/documents-sds aktualisiert. Bei verschlüsselten Scripten wurden der instrumentierte Code nicht mehr korrekt hochgeladen. (!457)

v4.5.5

- Verbindungsprüfung während der Testausführung fehlertoleranter gemacht. (!456)

v4.5.4

- Binden auf Typescript v5.4.* wegen fehlendem `implicitUseStrict` in neueren Versionen. (#64641)

v4.5.3

- Korrektur des Löschens von Testdaten, wenn Documents-Objekte mehr als einmal vorhanden waren. (#63857)

v4.5.2

- Ausgabeordner und Dateinamen der Coverage-Berichte hatten sich geändert. (!436)

v4.5.1

- Abhängigkeit `istanbul-lib-instrument` wurde wieder hinzugefügt.

v4.5.0

- Code-Coverage-Bibliothek istanbul auf v3 aktualisiert. (!425)
    - Neue Report-Formate: `text-summary`, `lcov`, `json`, `json-summary`, `teamcity`, `clover`
- CLI-Bibliothek commander auf v13 aktualisiert. Breaking Changes wurden deaktiviert, und kommen mit der v5.0.0 von otrTest. (!422)

v4.4.0

- Neue Funktionen zum Erzwingen der Testdatenlöschung, auch wenn dabei Fehler auftreten. Aktuell wird dies aber nur Fehler im OnDelete-Exit von Mappen unterstützt. (#63702)

```
desribe("Test data deletion", function() {
  // Für die gesamte Suite
  this.enforceTestDataDeletion();
  it("should delete test data", function() {
    // Nut für diesen Testfall
    this.enforceTestDataDeletion();
  });
});
```

v4.3.3

- Technisches Release zu v4.3.2.

v4.3.2

- Fehlerhafte Parameterübergabe führten durch Commander v13 zu Abbrüchen.

v4.3.1

- Aufräumen konnte mit `DlcErrFileUnknown` fehlschlagen, wenn eine Mappe in ihrem OnDelete-Exit andere Mappen gelöscht hat, die ebenfalls zum Aufräumem vorgemerkt waren. (!420)

v4.3.0

- Neue Funktion zum Deaktivierung der verteilten Testausführung in einer Testsuite. (#40)

```
describe("...", function() {
  // Deaktiviert die verteilte Testausführung für alle Tests dieser
  // Suite und garantiert, dass alle Tests dieser Suite vom gleichen
  // Runner ausgeführt werden.
  this.disableDistributedTestExecution();
});
```

- Fehler beim Einbinden von otrLogger auf Documents 5 korrigiert. (!407)

v4.2.0

- Neue Optionen `--no-backup`, `--no-forbid-only` und `--no-forbid-pending` für das `test`-Kommando, mit denen sich die dazugehörigen Optionen deaktivieren lassen.
- Testausführung schlug fehl, wenn in einem Testscript mit Mocha-Interface `context.returnValue` gesetzt wurde.
- Aktualisierung von @types/chai auf v4.3.20.

v4.1.1

- Mappen werden durch das Testframework beim Löschen in den Mappenpool verschoben. Andernfalls konnte das zu Abstürzen führen.
- `setBeforeDeleteTestDataCallback()` in Typings ergänzt.
- Aktualisierung von @types/chai auf v4.3.17.

v4.1.0

- Optimierung der parallelen Testausführung. Die kann mittels eines Webdis-Servers pro Testsuite parallelisiert werden. Wird über die folgenden neuen Optionen gesteuert. Siehe hierzu auch die Dokumentation (#18):
  - `--webdis-host`
  - `--webdis-execution-id`
  - `--webdis-executor-id`
  - `--shuffle-testsuites`
- Neue Funktion `UnitTest.captureLogs(capture: boolean)`, mit der die Lognachrichten, die während eines Unit Tests mit otrLogger geloggt werden, vom Testframework aufgezeichnet und im Fehlerfall mit ausgegeben werden.

  ```
  it("Retry case example", function() {
    this.captureLogs(true);
  });
  ```

  Hinweis: Setzt otrLogger v3.1.0 voraus.
- Funktionen zum Wiederholen von fehlschlagenden Testfällen. Das ist sinnvoll bei Wackeltests, die aufgrund von instabilen äußeren Faktoren fehlschlagen, und nicht aufgrund eines Fehler. Die `beforeEach`- und `afterEach`-Hooks werden dabei jedes mal ausgeführt. Ist in Anlehung an Mocha implementiert: https://mochajs.org/#retry-tests

```
describe("Retry example", function() {
  // Alle Testfälle, werden 2 wiederholt, wenn sie fehlschlagen
  this.retries(2);

  // Dieser Testfall, wird 4 wiederholt, wenn er fehlschlagen
  it("Retry case example", function() {
    this.retries(4);
  });
});
```

- Die Coverage Ergebnisse werden bei Verwendung des PrettyFormatter (Default bei manueller Ausführung, nicht über das CLI) nicht mehr ausgegeben. (!363)
- Optimierung der Übertragung der Testergbenisse vom Server. (#30)
- Neue Konfigurationsoption `testSuiteNamePattern`. Die kann verwendet werden, wenn die für die Testsuiten ein anderes Namensschema verwendet wird, wie z.B. ' *.test.js`. (#25)
- Default-Werte für `testSuiteDirectory` und `testCoverage` berücksichtigen in TypeScript-Projekten das Ausgabeverzeichnis, welche in der `tsconfig.json` angegeben ist. Für die Testsuite muss die `tsconfig.json` neben der Konfigurationsdatei `otr-test.json` liegen. (#27)
- Erweiterung der Angaben zu Dateinamen, Hostnamen usw. im JUnit-Bericht. (!364, !365)
- Erweiterung der Log-Ausgaben um Angaben zur Testsuite. (!366)

v4.0.1

- Aufrufe der alten `beforeAll()` und `afterAll()`-Hooks konnten zu folgenden Fehler führen:

```
TypeError: this.currentTestContainer is null
```

v4.0.0

- Umbenennung der Hooks `beforeAll()` und `afterAll()` an in `before()` bzw. `after()`. Die Benennung ist nun gleich mit Mocha. Die alten Namen `beforeAll()` und `afterAll()` werden aber weiterhin unterstützt.

- Korrektur der Ausführungsreihenfolge der Hooks. Die ist nun wie folgt:

  1. `before()`
  2. `beforeEach()`
  3. `afterEach()`
  4. `after()`. Zuvor wurden `before()` bzw. `beforeAll()` erst nach dem `beforeEach()` ausgeführt. Die Ausführungsreihenfolge ist damit auch gleich zu der von Mocha.

- Neue Funktion `this.setTestSuitesName()` zum Setzen des Namens einer Menge von Testsuiten. (!344)

```
describe("Example", function() {
  this.setTestSuitesName("Suites collection");
  // ...
});
```

Wird aber nur im JUnit-Format berücksichtigt. Führt hier zu folgender JUnit-Ausgabe:

```
<testsuites name="Suites collection"><testsuite name="Example"> ... </testsuite>
</testsuites>
```

Bei anderen Ausgabeformaten wird dieser Name ignoriert.

Relevant ist das für den Import der XML-Dateien in Test-Management-Tools, die darüber die XML-Dateien

- Default-Einstellungen von chai.js angepasst (#26):
  - `includeStack`: Per Default wird die Aufrufliste (Callstack) in die Ausgabe eingefügt.
  - `truncateThreshold`: Per Default wird die Begrenzung der Ausgabe von geprüften Werten bei Fehlern von 40 auf 100000 Zeichen erhöht. Angepasst werden können die in Testsuite wie folgt:

```
const chai = require("chai");
chai.config.includeStack = false;
chai.config.truncateThreshold = 0; // Unbegrenzt
```

- Die Option `--timeout` wird auch beim Herunterladen der Coverage-Daten berücksichtigt. (!349)

v3.8.1

- Die Schalter `--no-log-download` und `--tls` für das `test`-Kommando wiederhergestellt. (!331)

v3.8.0

- Neue Funktion `this.executeAsUser(login: string, callback: () => T): T`, die die übergebene Callback-Funktion im Kontext des übergebenen Benutzers ausführt und anschließend wieder zum aktuellen Skriptuser zurück wechselt. (!316)
- Neuer Schalter `--testsuite`, um eine einzelne Testsuite per `otr-test test` ausführen zu können. (!324)
- Neuer Schalter `--skip-test-init`, um die Testinitialisierung (Code Coverage Instrumenter, Upload der Sourcen, Ausführen von Hooks) zu überspringen. Nützlich in Kombination mit `--testsuite`. (!324)

- Das Testframework erkennt nun, wenn die Serververbindung getrennt wurde und bricht die Testausführung ab. (!325)
    - Neuer Schalter `--check-alive-interval <number>`, mit der die Wartezeit zwischen den Prüfungen eingestellt werden kann. Der Default-Wert ist 5 s.
- Korrektur der Beschreibung des Schalters `--timeout`. (!325)
- Das Testframework loggt nun auf der Konsole den Fortschritt der Testausführung. (!325)
- Das Löschen der Testdaten kann im SuperMode erfolgen. Die kann mit den neuen Funktionen `this.setDeleteTestDataInSuperMode(boolean): void` für eine Testsuite oder einem Testfall eingestellt werden. Per Default ist der SuperMode beim Löschen deaktiviert. (!318)
- Neuer Hook, der vor dem Löschen der Testdaten ausgeführt wird. Der kann mit `this.setBeforeDeleteTestDataCallback(callback: () => void): void` für eine Testsuite oder einen Testfall definiert werden. (!318)
- Kleinere Aktualisierung der Log-Ausgaben für die Hooks bei der Testausführung. (!319)
- Die Typings der Funktionen `this.util.findPortalScript`, `this.util.createCallbacks`, `this.util.createFellow` und `this.util.createSystemUser` waren nicht korrekt. (!330)

v3.7.0

- Neuer Schalter `--tls` zum Verbinden zu Documents per TLS. (#22)

v3.6.2

- Anpassung an Encoding-Breaking-Change in documentsOS. (#59127)

v3.6.1

- Die Kürzel von `--password` und `--port` waren identisch, was zu einem Konflikt führte. Neues Kürzel von `--password` ist `-P`. (!300)

v3.6.0

- Die Fehlermeldung bei Verwendung von `.only` und `.skip` in Kombination mit dem CLI-Parameter `--forbid-only` bzw. `--forbid-pending` wurde ist nun ausführlicher und schlägt nun mögliche Maßnahmen zur Behebung vor. (!284)
- Bei Verwendung des Pretty-Formatters (Default, wenn nicht per CLI ausgeführt) wird nun darauf hingewiesen `otr-test init` auszuführen, um Fehler durch fehlende Testdaten zu vermeiden, die ggf. in einem prelaunch-Hook angelegt werden. (!285)
- Die Ausgabe des Pretty-Formatters erfolgt nun in englischer Sprache. (!285)
- Die CLI-Flags `forbid-only` und `forbid-pending` werden als Mandanteneigenschaft gespeichert, damit das Testframework die Kommandozeilenparameter bei der Ausführung der

Tests auswerten kann. Die Mandanteneigenschaften wurden nach der Testausführung nicht wieder gelöscht. (!286)
- Bei Verwendung von `.only` wurde in der Serverkonsole fälschlicherweise die Anzahl aller Unit Tests der Testsuite geloggt anstatt nur die Tests, die per `.only` ausgeführt werden sollen (Meldung `Execute Unit Test X of X`). (!287)
- Aktualisierung von chai auf v4.4.0 (https://github.com/chaijs/chai/releases/tag/v4.4.0). (!289)

v3.5.3

- Die Typdefinition für `this.util` wurde korrigiert (Code Completion, Code Validierung).

v3.5.2

- Der Default-Wert von `--no-log-download` wurde korrigiert. Der hatte sich mit `v3.5.0` versehentlich geändert. (!269)
- Aktualisierung von @types/chai auf 4.3.11. (!270)

v3.5.1

- Der Code Instrumenter, der bei aktivierter Code Coverage mit dem Befehl `otr-test test` die JavaScript-Quelldateien instrumentiert, kann nun per `--no-instrument` deaktiviert werden. Hierdurch ist es bspw. möglich, die Instrumentierung des Codes von der Generierung des Coverage Reports zu trennen (bspw. bei Parallisierung der Testausführung). (!264)

v3.5.0

- Sämtliche CLI Optionen der Kommandos `test` und `coverage` können nun per Umgebungsvariable konfiguriert werden. Eine Liste der Umgebungsvariablen kann mit der Option `--help` des jeweiligen Kommandos abgefragt werden.
- Der Default-Wert der Option `--fetch-coverage-data` wurden an das Standardverhalten angepasst. (!259)

v3.4.1

- Die zusätzlichen Coverage Daten, die im Server erfasst werden, werden nun serverseitig komprimiert, um die Übertragungsdauer zu reduzieren

v3.4.0

- Der Schalter `--enforce-all-tests` kann nun alternativ per Umgebungsvariable `ENFORCE_ALL_TESTS` aktiviert werden.

v3.3.2

- Korrektur beim Aufsplitten von instrumentierten Quellcodedateien (!245, eb1465fd)

v3.3.1

- Beim Zusammensetzen der gesplitteten Dateien, die mehr als 256000 Zeichen haben, lag ein Syntaxfehler im Skript vor, das die gesplitteten Dateien wieder zusammen gesetzt hat. (d2679757)

v3.2.2

- Chai.js wurde beim Veröffentlichen nicht gebundelt; fehlten dann in der Registry. (!214)

v3.2.1

- Versionsnummer (`--version`) falsch, wenn als Abhängigkeit installiert (!212)

v3.2.0

Erweiterungen

- otrTest kann als Binärdatei ausgeliefert werden (!209, !211)
- Neuer Schalter `--no-coverage` für `test`, um die Code Coverage zu deaktivieren (!209)
- Neuer Schalter `--no-log-download` für `test`, um das Herunterladen der Server-Logs zu deaktivieren (!209)

v3.1.0

- Optimierungen für die Einbindung in andere Programme (#21)
- Neuer Schalter `--cwd` zum Setzen des Arbeitsverzeichnisses (!206)

v3.0.0

- Chai Assertion Library verfügbar (!185).
- Neuer Schalter `--enforce-all-tests` zur Ausführung in Matrix-Builds (#7).
- Konfig-Dateinamen in `otr-test.json` geändert. Umbenennung erfolgt automatisch, muss aber selbst eingecheckt werden (#4).
- Neue Konfigurationsoption `testSuiteDirectory`, mit der sich das Verzeichnis mit den Testsuiten konfigurieren lässt (!195).
- Umstellung des CLI-Parser (#10).
- Funktionen zusätzlich als API verfügbar (#10).
- Upgrade jsverify v0.8.4 (#8).
- Verbesserung der Doku (!183).

Getting started

Einrichten per Documents CLI

Die Initialisierung der Testumgebung ist in `documents deploy` integriert. Sofern der Schalter `--dev` übergeben wird, ruft `documents deploy` automatisch den Befehl `documents test init` mit passenden Verbindungsdaten zum documentsOS-Server auf.

1. Gewünschten Branch des Projekts auschecken
2. Entwicklungsmandant aufsetzen:
   `documents deploy --principal myDevPrincipal --host localhost --dev`
   (bzw. `npm run deploy`, wenn ein `scripts`-Kommando eingerichtet ist).

Einrichten in einem Projekt mit Tests

1. Gewünschten Branch des Projekts auschecken
2. Testframework importieren, weitere Sourcen und Hooks aus der `otr-test.json`:

   ```
   $> documents test init --host localhost --principal test
   ```

3. Per VS Code gewünschte Testsuite öffnen, hochladen und ausführen
4. Scripts in `package.json` ergänzen

   ```
   {
      scripts: {
         // ...
         "test": "documents test"
      }
   }
   ```

5. Ordner anlegen

```
mkdir src/test
```

6. otrTest konfigurieren in `src/test/otr-test.json` (siehe auch Konfiguration):

```
{
  // Skripts to upload before starting the test execution
   "include": [
     "src/test/util/**/*.{js,mjs}"
  ],
   "testCoverage": {
     "include": [
       "src/jscript/**/*.{js,mjs}",
       // any other needed source files
     ],
     "exclude": [
         // files the should excluded from the coverage
     ]
   }
 }
```

7. Ignore-Dateien erweitern (`.npmignore` und `.gitignore`):

```
# Instrumented source files and coverage files
instrumentedFiles/
coverage/
html-report/*
# Execution data
backup/
serverLogs.zip
# Test results
test.*.xml
```

8. Testsuite unter `src/test` anlegen, z. B. `src/test/test.HelloWorld.js`:

```
context.enableModules(undefined, 3); // Only needed on Documents 5
const { describe, it } = require("otrTest");
const { expect } = require("chai");

describe("Say Hello world", () => {
    it("should not fail, as Hello is always fine", function() {
        var say = "Hello World!";
      expect(say).to.be.equal("Hello World!");
    });
});
```

9. Tests initialisieren und ausführen

```
npm run test
```

Konfiguration

Die folgende Konfigurationsdatei muss unter `src/test/otr-test.json` abgelegt werden.

Minimalkonfiguration

```
{
      "include": [
            // Optional. Weitere Sourcen wie Hilfsbibliotheken, die für das Testen
            // benötigt werden. Für das Deployment des Testmandanten wird die
            // Verwendung von otr-build empfohlen.
            "src/test/util/**/*.{js,mjs}"
      ],
      "testCoverage": {
            "include": [
                  "src/jscript/**/*.{js,mjs}"
            ],
            "exclude": []
      }
}
```

Gesamtkonfiguration

Die Eigenschaften können nach Bedarf hinzugefügt oder entfernt werden:

```
{
	// Name des Mandanten, indem die Tests ausgeführt werden sollen (muss
	// selbst deployed werden)
	// Kommandozeilenargument `-principal`hat Vorrang
	"principal": "test",

	// Benutzer / Redakteur, der über ausreichend Rechte verfügt, um ggf.
	// Mandanten anzulegen
	// Optional (default = admin ohne Passwort). Benutzer / Redakteur für
	// die Skriptausführung per SDS (muss sich im Manager anmelden können)
	// Kommandozeilenargument `--user`und --password hat Vorrang
	"user": "admin",
	"password": "",

	// glob-Patterns zum Hochladen von Skripten und XML-Dateien vor dem Starten der Tests
	"include": [
		"src/test/util/**/*.{js,mjs}",
		"src/test/util/**/*.xml"
	],

	// Liste Testsuite-Dateinamen, die nicht generell ausgeführt werden sollen
	"exclude": [
		"test.term.matrix.termination.series" // <-- Dateiname der Testsuite
	],

	// Liste von Testsuiten, die in Merge Requests nicht ausgeführt werden sollen
	// (Wird nur angewendet, wenn Kommandozeilenargument `-mr`definiert wurde)
	"nonMergeRequestTestSuites": [
		"test.term.matrix.termination.series" // <-- Dateiname der Testsuite
	],

	"scripts": {

		// Wird noch vor dem Upload der Source Files ausgeführt
		"preinit": [
			{
				"type": "shell",
				"command": "npm run compile"
			}
		],
```

```
        // Deprecated. Wird zum gleichen Zeitpunkt wie preinit ausgeführt.
        "postinit": [],

        // Wird vor dem Upload der Sourcen in "config.include" hochgeladen
        "preimport": [],

        // Wird vor dem Ausführen der Tests ausgeführt.
        // Zu diesem Zeitpunkt wurden die Sourcen "config.include" bereits importiert
        "prelaunch": [
            {
                // Portal-Skript, welches bereits auf dem Server ist,
                // ausführen (z.B. Initialisierung)
                "type": "portalscript",
                "command": "otrTerm-INIT" // Name des Skripts
            }
        ],

        // Wird nach dem Ausführen der Tests aufgerufen
        "postlaunch": [
            {
                // Portal-Skript, welches bereits auf dem Server ist,
                // ausführen (z.B. Initialisierung)
                "type": "<cmd|shell|sh>",
                "command": "node byeByeWorld.js"
            }
        ]
    },

    // Wenn angegeben, werden Coverage-Daten gesammelt
    "testCoverage": {
        // Sourcen, für die Coverage-Daten gesammelt werden sollen
        "include": [
            "src/jscript/**/*.{js,mjs}"
        ],

        // Einzelne Ordner und Skripte, die ausgeschlossen werden sollen
        "exclude": [
            "src/jscript/otrTest_TypeDefs.js"
        ]
    },
```

```
      // Pfad zum Ordner, in denen sich die Testsuites befinden. Bei der Verwendung von
Transpilern, wie typescript,
      // kann der Ausgabepfad der transpilierten Testsuites abweichen.
      "testSuiteDirectory": "src/test"
}
```

CLI-Befehle und Parameter

Liste der verfügbaren Befehle

Mit dem folgenden Befehl können die verfügbaren Befehle der CLI ausgegeben werden. Jeder Befehl ist jeweils mit einer kurzen Beschreibung erläutert:
Hinweis: Für eine vollständige Liste bitte den Befehl manuell ausführen

```
$> documents test --help
Usage: documents test [options] [command]

Start execution of all unit tests

# [...]

Commands:
  init [options]                  Prepares the DOCUMENTS principal for the test execution.
                                  Uploads the test framework and other dependencies needed for
                                  the test execution and executes hooks defined in the
                                  configuration file otr-test.json.
  coverage                        Initialization and processing of the code coverage
```

Liste der verfügbaren Kommandozeilenparameter

Jeder Befehl unterstützt verschiedene Kommandozeilenparameter, die mit `--help` ausgegeben werden können:
Hinweis: Für eine vollständige Liste bitte den Befehl manuell ausführen

```
$> documents test --help
Usage: documents test [options] [command]

Start execution of all unit tests

Options:
  -m, --principal <string>        Id of the target principal
  -u, --user <string>             Login (fellow or admin) used to deploy the principal
                                  (env:DOCUMENTS_SERVER_USER) (default: "admin")
  -p, --password <string>         Password of the passed user (env:DOCUMENTS_SERVER_PASSWORD)
```

```
                                  (default: "")
  -H, --host <address>            Host / IP of the DOCUMENTS-Server (env:DOCUMENTS_SERVER_HOST)
                                  (default: "localhost")
  -p, --port <number>             Port of the DOCUMENTS-Server (env:DOCUMENTS_SERVER_PORT)
                                  (default: "11000")
  --coverage                      Enables code coverage (if configured). (default: true)
  --no-coverage                   Disables code coverage even when configured.
  --coverage-dir <string>         Path to the output folder for the coverage data (default:
                                  "coverage")
  --coverage-dir-prefix <string>  Name prefix of folders which contains coverage data. Will be
                                  used instead of 'coverage-dir' if defined. (default: "")
  --enforce-all-tests             Enforces the execution of all tests on all parallel nodes.
                                  Useful for matrix builds in pipelines (default: false)
  --formatter <name>              Name of a supported formatter for the test results (e. g.
                                  JUnitXMLFormatter, BoostTestXmlFormatter) (default:
                                  "JUnitXMLFormatter")
  --format <string>               Report format for the command `report`(comma separated list).
                                  Supported formats: html, cobertura, text (default: "html")
  --mr                            If specified only merge request test suites will be executed
                                  (default: false)
  --num-executors <number>        Number of executors for parallel test execution on different
                                  documents servers (env: CI_NODE_TOTAL) (default: 1)
  --executor-id <number>          Id of the executing node in case of parallel test executions
                                  (default: 1)
  --backup                        Specifies if a backup should be created after text execution
                                  (only if tests failed). (default: true)
  --forbid-only                   Enforce a rule that tests may not be exclusive (use of e.g.,
                                  describe.only() or it.only() is disallowed). (default: true)
  --forbid-pending                Enforce a rule that tests may not be skipped (use of e.g.,
                                  describe.skip(), it.skip(), or this.skip() anywhere is
                                  disallowed). (default: false)
  --timeout                       Timeout for each test suite in milliseconds (env:
                                  TESTSUITE_TIMEOUT)
  --no-log-download               Disables the download of the server logs
  -h, --help                      display help for command
```

Konfiguration über Umgebungsvariablen

Einige Kommandozeilenparameter können über Umgebungsvariablen konfiguriert werden. Als Beispiel sei hier die Verbindungsinformationen zum DOCUMENTS-Server genannt:

Umgebungsvariable	Beschreibung
DOCUMENTS_SERVER_HOST	Host / IP des Servers, auf dem die Tests ausgeführt werden sollen.
DOCUMENTS_SERVER_PORT	Port des Servers.
DOCUMENTS_SERVER_USER	Administrativer Benutzer oder Redakteur.

Umgebungsvariable	Beschreibung
DOCUMENTS_SERVER_PASSWORD	Passwort des Benutzers

Diese Liste ist nur ein **Auszug** der verfügbaren Umgebungsvariablen. Eine vollständige Liste kann mit `--help` hinter dem jeweiligen Befehl abgefragt werden. In der Beschreibung des Parameters wird der Name der Umgebungsvariable (sofern verfügbar) genannt (erkennbar an `env: <Name der Umbebungsvariable>`):

```
$ documents test --help
Usage: documents test [options] [command]

Start execution of all unit tests

Options:
  -u, --user <string>            Login (fellow or admin) used to deploy the principal
                                 (env:DOCUMENTS_SERVER_USER) (default: "admin")
  -p, --password <string>        Password of the passed user (env:DOCUMENTS_SERVER_PASSWORD)
                                 (default: "")
  -H, --host <address>           Host / IP of the DOCUMENTS-Server (env:DOCUMENTS_SERVER_HOST)
                                 (default: "localhost")
  -p, --port <number>            Port of the DOCUMENTS-Server (env:DOCUMENTS_SERVER_PORT)
                                 (default: "11000")
                                 documents servers (env: CI_NODE_TOTAL) (default: 1)
  --timeout                      Timeout for each test suite in milliseconds (env:
                                 TESTSUITE_TIMEOUT)
  ...
```

Mocha-Interface

Mocha ist ein weitverbreitetes JavaScript-Testframework. Dessen Interface wurde in otrTest implementiert, so dass Tests auch entsprechend der Mocha-Nomenklatur geschrieben werden können.

Die otrTest-Implementierung des Mocha-Interfaces hat jedoch folgende Einschränkung:

- `describe()` können nicht verschachtelt werden
- `it()` unterstützt keine Arrow-Funktion

Hallo-Welt-Beispiel

Wichtigsten Elemente eines Mocha-Tests sind die beiden Funktionen

- `describe()`: Umfasst mehrere Tests, die das gesamte Verhalten einer Funktionalität beschreiben, z. B. `describe("Create contract", ...);`.
- `it()`: Testet ein bestimmte Verhalten der Funktionalität, z. B. `it("should fail, if subject is missing", ...)`. Hier können Prüfungen implementiert werden. Schlagen die

fehl, dann müssen die eine Exception werfen. Es können aber hier die bekannten `this.assert`-Funktionen oder andere Expect/Assert-Funktionen verwendet werden.

Im einfachsten Fall sieht eine Testsuite wie folgt aus.

```
// #import [requireShim]

// Destructing assignment, um nicht immer die Funktionen
// in den lokalen Namensraum zu holen.
const { describe, it } = require("@otrTest/otrTest");

describe("Say Hello world", () => {
      it("should not fail, as Hello is always fine", function() {
            var say = "Hello World!";
            this.assertIsExactEqual(say, "Hello World!");
      });
      it("should also not fail, as it does nothing", function() {
            // Do nothing
      });
      it("should not fail, but does", function() {
            throw new Error("Fail!");
      });
});
```

Die kann normal ausgeführt werden. Das Ergebnis ist:

```
Say Hello world (0 Sek.)
      Erfolgreich: 2 von 3
      Fehlgeschlagen: 1 von 3

      o should not fail, as Hello is always fine (0 Sek.)
      o should also not fail, as it does nothing (0 Sek.)
      x should not fail, but does (0 Sek.)
            Error: Fail!
            Stack:
            ...
```

De-/Initialisierung der Testsuite

`beforeAll()` kann verwendet werden, um vor der Ausführung von Tests die notwendigen Vorbereitungen getroffen werden, wie z. B. die notwendigen Testdaten zu erzeugen.

Analog `afterAll()`, um nach der Ausführung der Testsuite aufzuräumen.

```
// #import [requireShim]

const {
      describe,
      it,
      beforeAll,
      beforeEach,
      afterAll,
      afterEach
} = require("@otrTest/otrTest");


describe("Say Hello world", () => {

      beforeAll(function() {
            util.out("Prepare to say 'Hello'");
      });

      beforeEach(function() {
            util.out("Prepare an it");
      })

      afterAll(function() {
            util.out("No cleanup, as nothing done");
      });

      afterEach(function() {
            util.out("After an it");
      });

      it("should not fail, as Hello is always fine", function() {
            var say = "Hello World!";
            this.assertIsExactEqual(say, "Hello World!");
      });

      it("should also not fail, as it does nothing", function() {
            // Do nothing
      });
});
```

Erkennbar ist das in der Ausgabe an den `preConditions` und `postCondition`:

```
Say Hello world (0 Sek.)
      Erfolgreich: 3 von 3
      Fehlgeschlagen: 0 von 3

      o preConditions (0 Sek.)
      o should not fail, as Hello is always fine (0 Sek.)
      o postConditions (0 Sek.)
```

Auszug aus dem Server-Log:

```
preConditions: SUCCESS
Prepare an it
After an it
Delete test data of test 'should not fail, as Hello is always fine'
should not fail, as Hello is always fine: SUCCESS
Execute unit test (2 of 2): should also not fail, as it does nothing
Prepare an it
After an it
Delete test data of test 'should also not fail, as it does nothing'
should also not fail, as it does nothing: SUCCESS
Prepare an it
No cleanup, as nothing done
After an it
Delete test data of test 'postConditions'
postConditions: SUCCESS
```

Testausführung steuern

Um nur einzelne Tests zu überspringen oder nur genau einen Tests auszuführen, stehen die beiden Funktionen

- `only()`: Nur dieser Test wird ausgeführt; alle anderen nicht
- `skip()`: Dieser Test wird nicht ausgeführt; alle anderen schon

zur Verfüfung. Geschrieben werden sie "hinter" die Funktionen `it()`.

only()

```
// #import [requireShim]

const { describe, it } = require("@otrTest/otrTest");

describe("Say Hello world", () => {
      it.only("I am the only one", function() {
            var say = "Hello World!";
            this.assertIsExactEqual(say, "Hello World!");
      });
      it("will be not executed", function() {
            // Do nothing
      });
});
```

Ausgabe:

```
Say Hello world (0,001 Sek.)
      Erfolgreich: 1 von 1
      Fehlgeschlagen: 0 von 1

      o I am the only one (0,001 Sek.)
```

`skip()`

```
// #import [requireShim]

const { describe, it } = require("@otrTest/otrTest");

describe("Say Hello world", () => {
	it.skip("I will be skipped", function() {
		var say = "Hello World!";
		this.assertIsExactEqual(say, "Hello World!");
	});
	it("I will be executed", function() {
		// Do nothing
	});
});
```

Ausgabe:

```
Say Hello world (0 Sek.)
	Erfolgreich: 1 von 1
	Fehlgeschlagen: 0 von 1

	o I will be executed (0 Sek.)
```

Löschen der Testdaten deaktivieren

otrTest löscht automatisch nach Abschluss eines Testfalls (sowohl im Erfolgs- als auch im Fehlerfall) alle Testdaten, die z. B. per `this.addDataToDelete` an otrTest übergeben wurden. Zu Debug-Zwecken kann es hilfreich sein, das Löschen der Testdaten zu deaktivieren:

```
it("will not delete my test data", function() {
	this.disableTestDataDeletion();

	var myFile = context.createFile("myFileType");
	this.addDocFileToDelete(myFile); // wird wegen this.disableTestDataDeletion() nicht
gelöscht
});
```

Sofern die Tests per CLI ausgeführt werden, wirft der Aufruf `this.disableTestDataDeletion()` eine Exception, um zu verhindern, dass der Aufruf versehntlich eingecheckt wird und damit die Ausführung andere Tests ggf. beeinflusst.

Testsuites

Testsuites müssen im Verzeichnis `src/test` abgelegt werden und mit das Präfix `test.` beginnen. Ein gültiger Name für eine Testsuite lautet z.B. `test.lcmContract_CoreAPI.js`:

```
src
  test
    lcmContract
      test.lcmContract_CoreAPI.js
  otrTest-config.json
```

Testcontainer

Jede Testsuite muss einen Testcontainer erstellen, denen die UnitTests (oder QuickCheckTests) zugeordnet werden. Ein Testcontainer wird wie folgt erstellt:

```
// #import [requireShim]

var otrTest = require("@otrTest/otrTest");
var testContainer = otrTest.getUnitTestContainer({
      name: "lcmContract_CoreAPI",                  // Name der Testsuite
      logPerf: true,                                // Loggt die Ausführungszeit der Tests
      formatter: new otris.test.JUnitXMLFormatter() // Optional. Formatter für die
Testergebnisse
 });
```

UnitTests

Erstellen eines UnitTests

Um einen Test zu erstellen, muss dieser dem `UnitTestContainer` hinzugefügt werden:

```
testContainer.addTest(new UnitTest(function() {
      this.assertIsTrue(false, "${value1} is not true");
}, "schreiber", "assert true is false"));
```

In diesem Beispiel wird der Test mit dem Benutzer `schreiber` ausgeführt. Der Test hat den Namen `assert true is false`. Das Testframework stellt eine Reihe von Assertion-Methoden bereit, die über `this.xyz` innerhalb des UnitTests aufgerufen werden können.
Auszug der verfügbaren Assertions:

- assertIsEqual / assertIsExactEqual: Prüfung auf Gleichheit (letztere mit Typgleichheit (== vs. ===)
- assertIsTrue / assertIsFalse
- assertIsDocFile
- assertReferenceFile
- assertIsSet

Das sind nur wenige der verfügbaren Assertions. Eine vollständige Liste kann am besten über die Code Completion von Visual Studio Code oder über die API-Dokumentation aufgerufen werden.

Ausführen von Tests

Eine Testsuite oder einzelne UnitTests können direkt durch das Ausführen des PortalScripts ausgeführt werden. Hierzu muss am Ende der Testsuite folgende Zeile ergänzt werden:

```
// Führt alle Tests aus
return testContainer.runAll();
```

oder

```
// Führt einen einzelnen Test aus
return testContainer.run("assert true is false");
return testContainer.run("assert true *");        // Führt alle Tests aus, dessen Namen dem
regulären Ausdruck entsprechen
```

Automatisches Löschen von Testdaten

Wenn die Funktionen des Testframeworks (z.B. `createDocFile`, `createFolder`...) verwendet werden, werden die so erzeugten Daten nach dem Ausführen des UnitTests automatisch wieder gelöscht.

Um auch Daten zu löschen, die durch externe Funktionen entstanden sind, können diese dem Testframework übergeben werden:

```
testContainer.addTest(new UnitTest(function() {
      /** @type {DocFile} */
        var contract = lcmContract_CoreAPI.create(...);
      this.addDocFileToDelete(contract.getid());       // Löscht nach dem Beenden des
UnitTests den erstellten Vertrag
}, "<Benutzer-Login>", "<Name des Tests>"));
```

Das Löschen der Testdaten kann auf mehrere Wege unterbunden werden, ohne den Testfall selbst anpassen zu müssen:
Deaktivieren am Testcontainer

```
var testContainer = otrTest.getUnitTestContainer({
      deleteTestData: false                                  // Löschen der Testdaten
deaktivieren
});
```

Deaktivieren am Unit-Test

```
testContainer.addTest(new UnitTest(function() {
    /** @type {DocFile} */
    var contract = lcmContract_CoreAPI.create(...);
    this.addDocFileToDelete(contract.getid());
}, "<Benutzer-Login>", "<Name des Tests>", false));           // 3. Parameter
```

Substitutionen / Pseudo-Templateliterale

Die Fehlermeldungen können bei Assertions durch Substitutionen erweitert werden, womit man auf die Eingabeparameter zugreifen kann, die verglichen werden. Aktuell werden die folgenden Substitutions unterstützt:

```
this.assertIsExactEqual(1, 2, "Values (${values}) are not equal"); // => "Values (1, 2) are
not equal"
this.assertIsExactEqual(1, 2, "${value1} is not equal to ${value2}"); // => "1 is not equal
to 2"
```

preConditions / postConditions

Am Testcontainer können so genannte preConditions bzw. postConditions definiert werden. Das sind Funktionen, die vor bzw. nach dem Ausführen der UnitTests des Testcontainers aufgerufen werden.

Technisch gesehen handelt es sich dabei ebenfalls um einen UnitTests, weshalb innerhalb der Funktion sämtliche Funktionen des UnitTests über das `this`-Objekt verfügbar sind:

```
testContainer.setPreConditions(function() {
    // z. B. Erstellen von Testdaten
}, "schreiber");

testContainer.setPostconditions(function() {
    // z. B. aufräumen, nachträgliche Auswertung etc.
}, "schreiber");
```

Auch innerhalb der preConditions können Testdaten per `this.addDocFileToDelete` zum Löschen nach dem Ausführen **aller** UnitTests definiert werden.

Ausführungsablauf

Die Tests werden wie folgt ausgeführt:

1. Ausführen der preCondtions
2. Ausführen aller UnitTests. Die Reihenfolge ist dabei nicht definiert (UnitTests sollten keine Voraussetzungen / Abhängigkeiten zu anderen UnitTests aufweisen).
3. Ausführen der postConditions

Ein UnitTest wird wie folgt ausgeführt:

1. Wechsel zum angegebenen Skriptuser (hier: `schreiber`)
2. Ausführen der Funktion, die dem UnitTest übergeben wurde. Sollte die Funktion eine Exception werfen, wird diese vom Testframework abgefangen und die Ausführung des nächsten Tests begonnen.
3. Nach dem ein UnitTest (mit oder ohne Fehler) beendet wurde, werden alle Testdaten in umgekehrter Reihenfolge gelöscht (First in, last out).

Vollständiges Beispiel

```
var otrTest = require("@otrTest/otrTest");
var testContainer = otrTest.getUnitTestContainer({
      name: "lcmContract_CoreAPI",                          // Name der Testsuite
      logPerf: true,                                        // Loggt die Ausführungszeit der
Tests
      formatter: new otris.test.JUnitXMLFormatter(),        // Formatter für die
Testergebnisse
            appendStacktraceForUnitTests: true,                   // Gibt den Stacktrace in
der Fehlermeldung des Testfalls aus
});

testContainer.addTest(new UnitTest(function() {
      this.assertIsTrue(false, "${value1} is not true");
}, "schreiber", "assert true is false"));

// Führt alle Tests aus
return testContainer.runAll();
```

QuickCheck-Tests

Was genau QuickCheck-Tests sind, könnt ihr hier nachlesen. Das Testframework hat die Bibliothek jsverify eingebunden. Die Typings für VS Code wurden entsprechend erweitert und eingebunden, sodass die volle Code-Completion zur Verfügung steht.

Um einen QuickCheck-Test anzulegen, geht man wie folgt vor:

```
// #import [requireShim]
const QuickCheckTest = require("@otrTest/test.otrQuickCheckTest");
const TestOutput = require("@otrTest/test.otrUnitTestOutput");

var otrTest = require("@otrTest/otrTest");
var testContainer = otrTest.getUnitTestContainer({
    name: "QuickCheckExample",                  // Name der Testsuite
    logPerf: true,                                // Loggt die Ausführungszeit der Tests
    formatter: new TestOutput.JUnitXMLFormatter() // Optional. Formatter für die
Testergebnisse
 });

testContainer.addTest(new QuickCheckTest(function(jsc) {
      // Stellt sicher, dass die Wurzel einer zufällig quadrierten Zahl
      // der Ausgangszahl entspricht.
      var quickCheckFunction = this.bind(function(randomNumber) {
            return Math.sqrt(Math.pow(randomNumber, 2)) === randomNumber;
      });

      // Führt den Test 100! mal aus, mit zufällig generierten Eingabeparametern
      jsc.assertForall(jsc.nat, quickCheckFunction);
}, "<Benutzer-Login>", "<Name des Tests>"));
```

Der Unterschied zum Erzeugen eines UnitTests ist der Klassenname (`UnitTest` vs. `QuickCheckTest`). Viel wichtiger ist, dass der auszuführenden Funktion (der 1. Parameter des Klassen-Konstruktors) ein Parameter übergeben wird, über den die Funktionen der jsverify-Bibliothek verwendet werden können.

Chai.JS

Chai.JS ist eine Bibliothek, mit der in Tests Prüfungen implementiert werden können. Diese steht neben den nativ von otrTest bereitgestellten Prüffunktionen zur Verfügung.

- `expect/should` für Behavior-Driven Development
- `assert` für Test-Driven Development

Im Ergebnis verhalten sich beide Funktionen gleich, unterscheiden sich aber in der Formulierung der Prüfbedingung.

Für Details siehe die API-Dokumentation von Chai.JS.

Behavior-Driven Development

```
context.enableModules();
const { expect, should } = require("chai");

const a = 1;
expect(a).to.be.equal(1);

function f() {
      return 1;
}
should().not.throw(f);
```

Test-Driven Development

```
context.enableModules();
const { assert } = require("chai");

const a = 1;
assert.equal(a, 1);

function f() {
      return 1;
}
assert.doesNotThrow(f);
```

Tipps und Tricks

Testausführung beenden

Die Ausführung einer Testsuite kann abgebrochen werden, indem das Property `otrTestCancelTestExecution` im PropCache gesetzt wird:

```
propCache.otrTestCancelTestExecution = true;
```

Ein bereits laufender Test wird noch zuende ausgeführt. otrTest beendet die Ausführung mit dem nächsten Test der Testsuite und löscht das Property wieder aus dem PropCache.

Der propCache-Wert kann bspw. schnell und einfach im Skriptausführungsfenster mit der Serverkonsole der DOCUMENTS-Toolbox gesetzt werden.

Coverage-Daten von Gadgets-, Exits-, ScriptCall-Scripten und globale return-Statements

Werden Script im Rahmen von Tests nicht direkt ausgeführt, sondern z. B. durch ScriptCalls oder durch ausgelöste Exits, werden deren Coverage-Daten erfasst. Skripte mit globalem `return`-Statement werden ebenfalls nicht erfasst, da globale `return`-Statements nach ECMA-Standard syntaktisch falsch sind und vom Code Instrumenter übersprungen werden.

Um die Coverage-Daten solcher Scripte zu erfassen, kann das Script `collect-coverage-data` in diesen Scripten importiert werden. Das sorgt dafür, dass die Coverage-Daten in einem CustomProperty gespeichert werden, so dass Testframework die Daten herunterladen kann.

Zusätzlich ist es notwendig, globale return-Statements auf `context.returnValue` umzustellen.

Beispiel:

```
context.returnType = 'html';
// #import [collect-coverage-data]
context.returnValue = 'Alarm'; // statt return 'Alarm!';
```

Per Default lädt das Testframework die Daten herunter. Kann aber auch mittels des Schalters `--fetch-coverage-data` gesteuert werden.